Token Pattern Based AI Text Detection

Individual Project

Author: Priyanshu Sharma

Problem Description

Given a document, I must classify it as A (human written) or B (machine written). The training set given has 10,536 labelled documents, and the test set has 3,000 unlabelled documents. Each document consists of a sequence of integer token identifiers from a fixed vocabulary, and the original words cannot be recovered. With the help of various token statistics and patterns, I can classify the document into either A or B. My aim is to build a model that can predict the category of documents.

1. Project Overview

The goal of this project was to develop a machine learning system which classifies text into human written (A) or machine written (B). I had a dataset of 10,536 labelled training documents represented using token IDs, and 3,000 unseen documents for final prediction. I developed an approach that uses lexical patterns, statistical writing style features, and language model based predictability features in a multi-level stacking architecture.

I achieved 95.3% accuracy and 0.988 AUC on the unseen documents. I also used out-of-fold predictions and nested cross-validation in the system to reduce information leakage and overfitting.

The dataset includes 35.1% human-written and 64.9% machine-written documents. Since a classifier that predicts the majority class would already achieve 64.9% accuracy, I also used AUC as an evaluation metric.

I found through exploratory analysis that human-written and machine-written text have different characteristics. Machine documents are longer on average, with a mean of 213.2 tokens compared with 137.5 tokens for human documents. Machine text also contains more repeated three-token phrases and slightly longer sentences. Human text has a higher rate of rare words and more immediate word repetition. However, these individual features are weak predictors when used independently.

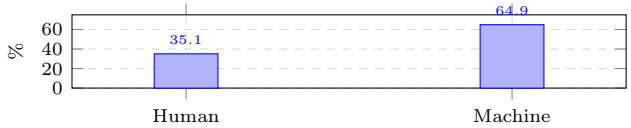


Figure 1: Training-set class distribution.

2. Dataset and Exploratory Analysis

The dataset includes 35.1% human-written and 64.9% machine-written documents. Since a classifier that predicts the majority class would already achieve 64.9% accuracy, I also used AUC as an evaluation metric.

I found through exploratory analysis that human-written and machine-written text have different characteristics. Machine documents are longer on average, with a mean of 213.2 tokens compared with 137.5 tokens for human documents. Machine text also contains more repeated three-token phrases and slightly longer sentences. Human text has a higher rate of rare words and more immediate word repetition. However, these individual features are weak predictors when used independently.

3. Individual Features

Individual statistical features do not perform particularly well at distinguishing human-written and machine-written text. The best individual feature I observed was the out-of-vocabulary (OOV) rate, which achieved an AUC of 0.615. Average sentence length achieved an AUC of 0.598, while variation in word rarity achieved an AUC of 0.597.

Average word rarity, variance of word frequency rank, and repeated three-token phrases each achieved an AUC below 0.60. I found that no single statistical feature was sufficient for reliable classification. Therefore, I combined multiple independent feature sets containing lexical, stylistic, and sequential information.

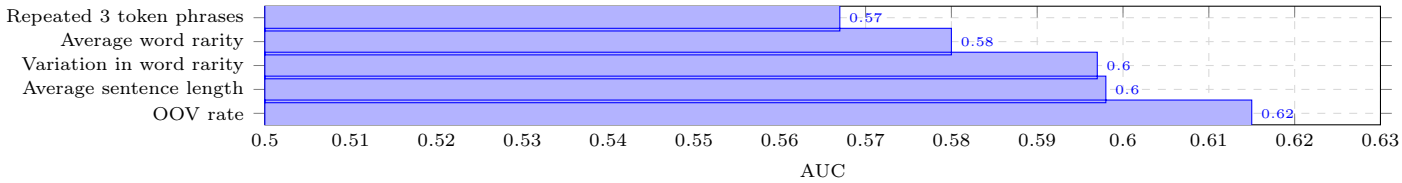


Figure 2: AUC of selected individual statistical features.

4. TF-IDF Features

The first feature representation I explored was TF-IDF using unigrams, bigrams, and trigrams, resulting in a large sparse feature space containing 349,372 features.

TF-IDF reduces the importance of terms that appear in many documents while highlighting more distinctive terms. I also included positional features representing the first two tokens, the last token, document length, and whether the document ends with a full stop.

I found that these features captured useful differences between human-generated and machine-generated text. Since the resulting feature space was large and sparse, I used linear models in the first layer for computationally efficient processing.

5. Statistical and Language Features

I extracted 30 statistical features to characterize writing style. These features included document length, vocabulary diversity, rare-word usage, repeated phrases, sentence length, word rarity, and variation across sentences and local windows.

I also found that variance-based features were useful because two classes can have similar averages but different distributions.

I trained separate human and machine language models using bigrams and trigrams. For each token, I calculated the log-likelihood ratio, defined as the difference between machine-model and human-model log-likelihoods.

At the document level, I constructed 25 features representing overall predictability and changes in predictability throughout the document.

6. Proposed Architecture

I designed a multi-level stacking architecture that combines lexical, statistical, and language-model features.

At Level 1, I used several expert models to generate prediction scores. At Level 2, I combined these scores using Logistic MLP, RBF SVM, and Gradient Boosting. At Level 3, I used Logistic Regression as a final judge to combine the Level-2 predictions and produce the final machine-written probability.

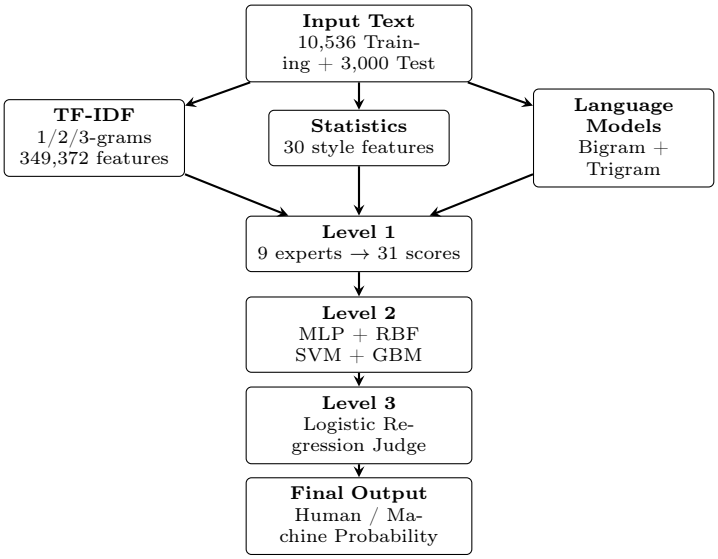


Figure 3: Multi-level stacking architecture used for human-vs-machine text detection.

7. Level-1 Expert Models

I used Logistic Regression, NB-weighted Logistic Regression, a Multi-Layer Perceptron, three Linear Support Vector Machine variants, Bigram Language Models, and Trigram Language Models.

Together, these models generated 31 prediction scores per document. Linear SVM was the strongest individual expert, achieving approximately 88.6% accuracy.

Although the language-model experts performed less effectively on their own, they captured word order and predictability information that phrase-based models could not fully represent. I used this diversity to improve the stacking architecture, as an ensemble benefits when its constituent models make different errors.

The Level-1 stage transformed a large feature representation into 31 prediction scores for the subsequent layers.

8. Out-of-Fold Training

I used out-of-fold training to reduce information leakage. I divided the training dataset into five folds. Each model was trained on four folds, while the remaining fold was used to generate predictions for documents that the model had not seen during training.

I repeated this process until every training document had received an out-of-fold prediction. These predictions were then passed to the next stacking layer.

For the 3,000 test documents, I used the five trained models to generate predictions and averaged their outputs.

This procedure provided more realistic inputs for the upper layers and reduced the risk of overfitting caused by using predictions from models that had already seen the same training documents.

9. Level-2 Models

I evaluated six candidate models at Level 2. Logistic MLP achieved 0.9495 accuracy and 0.9863 AUC. RBF SVM achieved 0.9491 accuracy and 0.9851 AUC. Gradient Boosting achieved 0.9494 accuracy and 0.9873 AUC.

Extra Trees, Random Forest, and Logistic Regression performed slightly worse. Therefore, I selected the three strongest and complementary models for the final ensemble.

The MLP learned nonlinear combinations of features, RBF SVM captured similarities in the feature space, and Gradient Boosting constructed decision trees sequentially. Their different learning mechanisms made them suitable candidates for stacking.

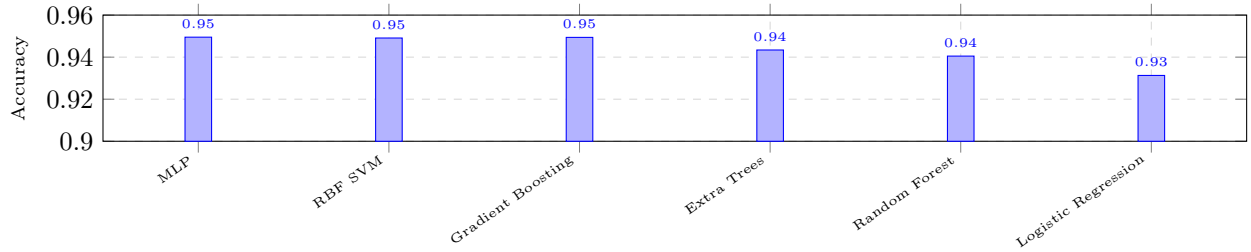


Figure 4: Accuracy of the Level-2 candidate models.

10. Final Judge

I used Logistic Regression as the Level-3 model to combine the predictions of the three selected Level-2 models. This model learns how to combine the outputs of the MLP, SVM, and Gradient Boosting models to estimate the final probability that a document is machine-written.

I selected a relatively simple final model to limit its capacity to overfit the training data. Nested cross-validation produced an accuracy of 0.9529.

I chose this approach because the lower-level models already contained many parameters. Rather than introducing another complex model, I used a simple final classifier to combine their complementary information and reduce the risk of overfitting.

11. Ablation Study

I conducted an ablation analysis to evaluate the contribution of the main system components.

TF-IDF Logistic Regression achieved 0.880 accuracy. Adding text models increased accuracy to 0.890. Statistical and variance features increased it to 0.913, while bigram language models increased it to 0.918.

Combining text, language-model, and statistical features increased accuracy to 0.940. Adding trigram and positional features raised it to 0.948. Incorporating Linear SVM scoring increased it to 0.950.

My final model, which combines MLP, SVM, and Gradient Boosting, achieved 0.953 accuracy. These results indicate that combining different types of information improved classification performance.

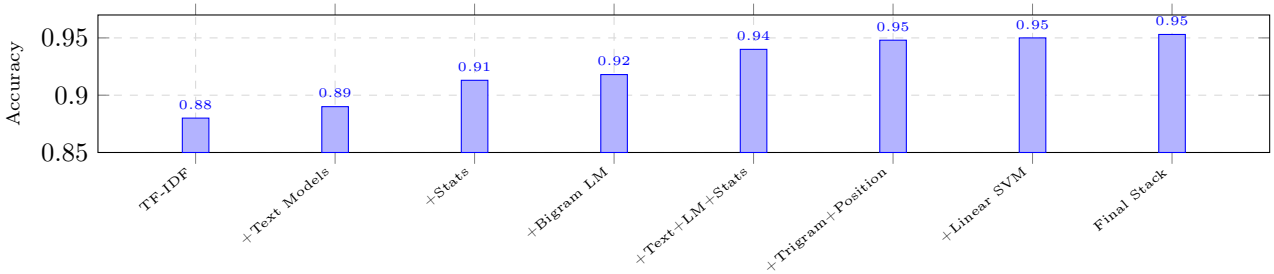


Figure 5: Ablation analysis of the major system components.

12. Final Performance

My final stacked model achieved an accuracy of 0.9529 and an AUC of 0.988 on unseen documents.

For comparison, Linear SVM achieved 0.886 accuracy and 0.952 AUC, while the Logistic MLP, RBF SVM, and Gradient Boosting models each achieved approximately 94.9% accuracy.

The final stacked model performed better than the individual Level-2 models. AUC measures how effectively a model ranks machine-written documents above human-written documents across classification thresholds.

The AUC of 0.988 indicates strong discrimination between the two classes. I therefore used AUC alongside accuracy to evaluate the final model.

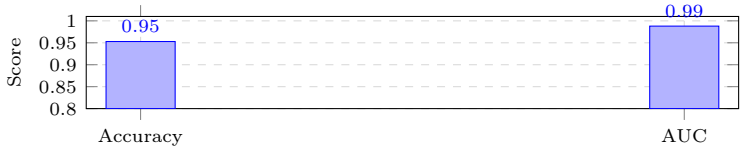


Figure 6: Final accuracy and AUC on unseen documents.

13. Validation and Generalization

I evaluated the model using five-fold cross-validation, training-validation gap analysis, nested cross-validation, AUC, and comparisons between training and test feature distributions.

The nested and non-nested final scores were 0.9529 and 0.9530, respectively. This small difference suggests that the final combination layer introduced little additional overfitting under the evaluated validation setup.

I also observed that increasing the amount of training data improved accuracy on unseen documents while reducing the training-validation gap.

I used Kolmogorov-Smirnov (K-S) tests to compare training and test feature distributions. These tests did not reveal large differences, supporting the possibility that the test data followed a similar distribution.

14. Limitations and Conclusion

One limitation I observed was the effect of document length. For documents containing 50 tokens or fewer, the error rate increased to approximately 7%. For very long documents containing more than 400 tokens, it dropped below 0.5%.

Human-written documents were classified slightly more accurately than machine-written documents, with error rates of approximately 5.7% and 4.3%, respectively.

The results may represent an optimistic estimate because different data folds were used at different stages of the project. This limitation should be considered when interpreting the reported performance.

Overall, I developed a human-versus-machine text classification system that achieved approximately 95.3% accuracy and 0.988 AUC. I combined TF-IDF features for lexical information, statistical features for writing style, and language-model predictability features in a stacked ensemble.

This project demonstrates how combining complementary feature representations and classifiers can improve classification performance over individual models.

Final Result: 95.3% Accuracy 0.988 AUC

References and Acknowledgements

1. **Claude, Anthropic.** Used for assistance with code generation, implementation support, debugging, and LaTeX report formatting.
2. **Punit Rathode.** Course lectures and instructional material used for understanding concepts covered in the course.
3. **Andrew Ng.** Course lectures and instructional material used for machine learning modeling concepts and model development.
4. **DeepLearning.AI.** Educational resources and research-oriented material consulted for machine learning and deep learning concepts.
5. **Google Gemini.** Used as a supplementary research and information resource during the development of the project.